

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Реализация метода Гаусса с выбором ведущего элемента для решения систем линейных уравнений»

Выполнил:

студент группы 3825Б1-ПМии1
Соснов М. С.

Проверила:

Бусько П.В.

Содержание

Введение	3
Постановка задачи	4
Руководство пользователя	5
Описание программной реализации.....	7
Результаты экспериментов	12
Заключение	18
Литература	19
Приложение	20

Введение

Решение систем линейных алгебраических уравнений (СЛАУ) является одной из фундаментальных задач вычислительной математики. В матричной форме такая система записывается как:

$$Ax=b$$

где A – матрица коэффициентов, b – вектор правой части, а x – искомый вектор неизвестных. Такие системы возникают во многих прикладных областях.

Одним из наиболее распространённых методов решения СЛАУ является метод Гаусса – алгоритм последовательного исключения неизвестных. Суть метода заключается в приведении расширенной матрицы системы к единичному виду с помощью элементарных преобразований строк. После этого значения неизвестных находятся естественным образом.

Математические объекты «вектор», «матрица» и «СЛАУ» выстраиваются в иерархию: матрица может рассматриваться как набор векторов, а СЛАУ – как матрица с методом решения. Такой подход позволяет эффективно использовать механизмы объектно-ориентированного программирования, в частности наследование классов.

Основная задача – продемонстрировать применение механизма наследования при разработке программы для решения СЛАУ методом Гаусса с выбором ведущего элемента

Постановка задачи

Целью работы является разработка программы для решения систем линейных уравнений методом Гаусса с выбором ведущего элемента. Программа должна быть реализована на языке C++ с использованием наследования и шаблонов классов.

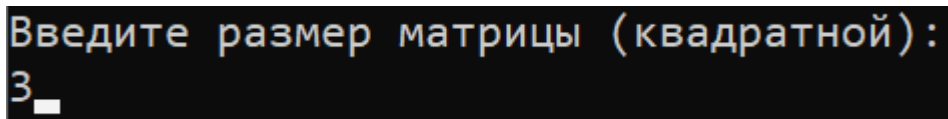
Необходимо было выполнить следующие задачи:

1. Реализовать шаблонный класс вектор.
2. Реализовать класс квадратная матрица, который является наследником шаблонного «вектора векторов».
3. Реализовать класс СЛАУ, который является наследником квадратной матрицы и у него есть метод Гаусса. Метод Гаусса должен принимать правую часть в качестве аргумента и возвращать ответ в виде вектора.

Руководство пользователя

Программа предназначена для решения систем линейных уравнений методом Гаусса с выбором ведущего элемента. Работа программы осуществляется через консольный интерфейс, ввод данных выполняется с клавиатуры.

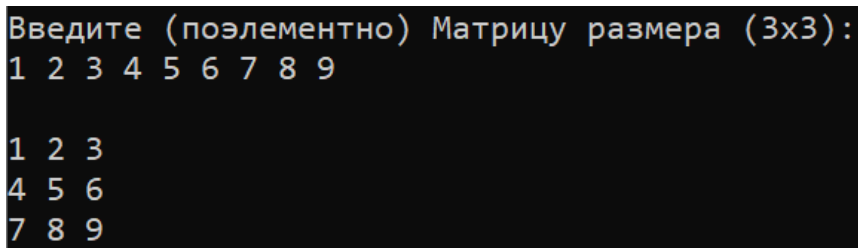
1. После запуска программы пользователю предлагается ввести размер квадратной матрицы системы. Вводится одно целое число – количество строк и столбцов матрицы.



```
Введите размер матрицы (квадратной):  
3
```

Рис. 1. Пример ввода размера матрицы

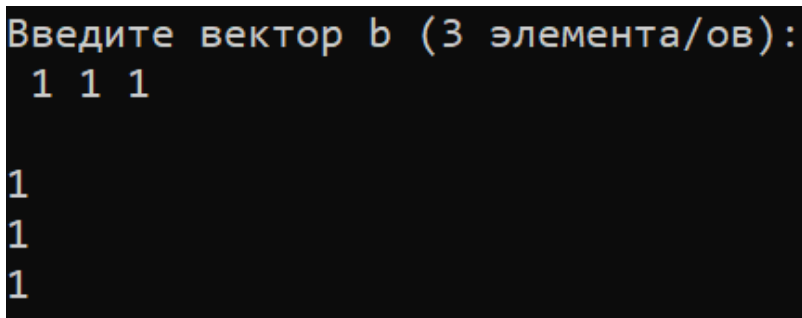
2. Далее пользователю предлагается последовательно ввести элементы матрицы, заданного размера. Ввод выполняется поэлементно, через пробел. После ввода матрица выводится на экран.



```
Введите (поэлементно) Матрицу размера (3x3):  
1 2 3 4 5 6 7 8 9  
  
1 2 3  
4 5 6  
7 8 9
```

Рис. 2. Пример ввода элементов матрицы

3. Затем необходимо ввести вектор правой части системы. Ввод выполняется поэлементно, через пробел. После ввода вектор также отображается на экране.



```
Введите вектор b (3 элемента/ов):  
1 1 1  
  
1  
1  
1
```

Рис. 3. Пример ввода вектора правой части

4. После получения всех исходных данных программа выполняет решение системы методом Гаусса с выбором ведущего элемента. Если матрица является невырожденной, на экран выводится вектор решения системы. В случае, если матрица вырождена решение системы не единственно или его не существует, программа выводит соответствующее сообщение об ошибке. После чего программа завершает свою работу.

```
Ответ:  
-0.714286 0.428571 0.285714
```

Рис. 4. Пример вывода решения системы

Описание программной реализации

Программа реализована на языке C++ и предназначена для решения систем линейных уравнений методом Гаусса с выбором ведущего элемента. В основе программной реализации лежит объектно-ориентированный подход, использование шаблонов классов и механизма наследования. Программа состоит из нескольких файлов: *Vector.h*, *Matrix.h*, *SLU.h* и *LabaratoryMatrix.cpp*.

С алгебраической точки зрения система линейных уравнений записывается в виде: $Ax = b$, где A – квадратная матрица коэффициентов системы, x – вектор неизвестных, b – вектор правой части.

Для нахождения решения используется метод Гаусса. Данный метод основан на последовательном выполнении элементарных преобразований строк матрицы. В результате этих преобразований исходная система приводится к эквивалентной системе более простого вида, из которой значения неизвестных могут быть найдены непосредственно.

В данной работе реализован метод Гаусса с выбором ведущего элемента. Такой подход используется не случайно. При вычислениях с числами с плавающей точкой возможно накопление ошибок округления. Если на очередном шаге деление выполняется на очень маленький элемент, то даже незначительная ошибка округления может многократно увеличиться и привести к существенному искажению результата. Именно поэтому перед каждым шагом исключения выбирается максимальный по модулю элемент текущего столбца. Использование ведущего элемента уменьшает влияние вычислительных ошибок.

Vector.h

Файл *Vector.h* содержит реализацию шаблонного класса *Vector<T>*, предназначенного для хранения и обработки одномерных массивов данных. Использование шаблонов позволяет создавать векторы для различных типов данных без дублирования кода. В данной работе используется тип *double*, однако класс остаётся универсальным. В классе содержатся два основных поля:

- *vec* – указатель на динамический массив элементов
- *size* – размер вектора

Использование динамической памяти позволяет задавать размер массива во время выполнения программы, что особенно важно при работе с системами линейных уравнений произвольного размера.

В классе реализованы следующие методы:

- *Vector()* – конструктор по умолчанию, создаёт пустой вектор.
- *Vector(int _size, T value = T())* – основной конструктор, выделяет память под массив заданного размера и заполняет его начальными значениями.
- *Vector(const Vector& copy)* – конструктор копирования.
- *Vector& operator=(const Vector& rhs)* – оператор присваивания.
- *T& operator[](int index)* – перегрузка оператора *[]*.
- *FillVector()* – ввод элементов вектора.
- *PrintVector()* – вывод элементов вектора строкой.
- *PrintVector2()* – вывод элементов вектора столбцом.
- *GetSize()* – получение размера вектора.
- *~Vector()* – деструктор.

Поскольку класс использует динамическое выделение памяти, особую роль играет корректная реализация конструктора копирования и оператора присваивания. Без их реализации происходило бы поверхностное копирование указателей, что привело бы к ошибкам при освобождении памяти. В реализованной версии создаётся полноценная копия массива с выделением новой области памяти. Для удобства работы перегружен оператор *[]*, благодаря этому доступ к элементам выполняется в привычной форме – *v[i]*. Освобождение динамической памяти выполняется в деструкторе, что предотвращает утечки памяти.

Matrix.h

Файл *Matrix.h* содержит реализацию шаблонного класса *Matrix<T>*, предназначенного для хранения квадратных матриц. Класс *Matrix<T>* является наследником шаблонного класса «вектор векторов» (*Vector<Vector<T>>*). Это означает, что матрица представляется как набор строк, где каждая строка является объектом класса *Vector<T>*.

В классе реализованы следующие методы:

- *Matrix(int n)* – конструктор, создаёт квадратную матрицу размера $n \times n$, заполненную нулевыми значениями.

- *FillMatrix()* – выполняет ввод элементов матрицы. Внутри метода для каждой строки вызывается функция заполнения вектора, реализованная в базовом классе.
- *PrintMatrix()* – выводит матрицу на экран, последовательно выводя каждую строку матрицы.
- *GetSize()* – возвращает размер матрицы.

Таким образом, класс *Matrix<T>* представляет собой расширение класса вектора и реализует хранение двумерной структуры данных на основе уже существующего класса.

SLU.h

Файл *SLU.h* содержит реализацию шаблонного класса *SLU<T>*, предназначенного для решения систем линейных уравнений. Класс *SLU<T>* наследуется от класса матриц, т.к. система линейных уравнений содержит матрицу коэффициентов и должна обладать всеми возможностями класса матрицы.

Основной частью класса является метод *Gauss()*, реализующий метод Гаусса с выбором ведущего элемента. Метод принимает вектор правой части системы и возвращает вектор решения. В начале работы создаётся копия матрицы системы и копия вектора правой части, это необходимо для сохранения исходных данных, поскольку метод Гаусса изменяет элементы матрицы в процессе вычислений.

Метод Гаусса основан на элементарных преобразованиях строк, цель алгоритма — привести матрицу системы к единичной при котором вектор правой части автоматически становится вектором решения системы. На каждом шаге алгоритма выбирается ведущий элемент. Для этого ищется максимальный по модулю элемент текущего столбца. Выбор ведущего элемента необходим для повышения точности вычислений, если выполнять деление на очень маленькие числа, то ошибки округления начинают быстро накапливаться, а промежуточные значения могут становиться очень большими. Использование максимального по модулю элемента уменьшает вероятность подобных ситуаций и делает метод более точным.

После выбора ведущего элемента выполняется перестановка строк матрицы и соответствующих элементов вектора правой части, перестановка строк является элементарным преобразованием и потому не изменяет решение системы.

Далее все элементы строки делятся на ведущий элемент, в результате чего диагональный элемент становится равным единице.

Следующий этап – исключение переменной из остальных строк. Для этого вычисляется коэффициент, после чего из каждой строки вычитается ведущая строка, умноженная на этот коэффициент. Такая операция соответствует элементарным преобразованиям строк и приводит к занулению элементов текущего столбца.

Аналогичные преобразования выполняются и для вектора правой части системы.

Особое внимание уделяется проверке вырожденности матрицы. Если ведущий элемент оказывается равным нулю, это означает, что определитель матрицы также равен нулю ($\det A = 0$). Такая система либо несовместна, либо имеет бесконечно много решений. В этом случае программа выводит сообщение об ошибке и возвращает пустой вектор.

LabaratoryMatrix.cpp

Файл *LabaratoryMatrix.cpp* содержит основную функцию `main()` и отвечает за взаимодействие с пользователем. В файле реализованы: ввод размера матрицы, ввод элементов матрицы коэффициентов и вектора правой части системы, вывод найденного вектора неизвестных, если решения не существует – выводится сообщение о вырожденности матрицы.

Оценка вычислительной сложности

Следует отметить, что метод Гаусса относится к численным методам с высокой вычислительной сложностью. Основная часть вычислений связана с последовательным выполнением элементарных преобразований строк матрицы.

При решении системы размера $n \times n$ количество арифметических операций пропорционально:

$$O(n^3)$$

Это связано с тем, что для каждого шага исключения требуется выполнять обработку всех оставшихся строк и столбцов матрицы.

Объем используемой памяти определяется хранением матрицы коэффициентов и дополнительных векторов, поэтому затраты памяти имеют порядок:

$$O(n^2)$$

Несмотря на достаточно большое количество вычислений, метод Гаусса остаётся одним из наиболее распространённых способов решения систем линейных уравнений благодаря универсальности и возможности получения точного решения за конечное число шагов.

Результаты экспериментов

В ходе выполнения лабораторной работы был проведён ряд вычислительных экспериментов, позволяющих проверить корректность работы программы, исследовать особенности метода Гаусса и проанализировать влияние вычислительных ошибок при работе с числами с плавающей точкой.

Эксперименты проводились для различных типов систем линейных уравнений:

- систем с невырожденной матрицей
- систем с вырожденной матрицей
- систем с малыми коэффициентами
- почти вырожденных систем

Особое внимание уделялось точности вычислений и роли выбора ведущего элемента.

Решение системы с невырожденной матрицей

Для проверки базовой корректности работы программы была рассмотрена система:

$$\begin{cases} 2x_1 + x_2 - x_3 = 8 \\ -3x_1 - x_2 + 2x_3 = -11 \\ -2x_1 + x_2 + 2x_3 = -3 \end{cases}$$

В этой системе матрица коэффициентов и вектор правой части выглядит следующим образом:

$$A = \begin{pmatrix} 2 & 1 & -3 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \quad b = \begin{pmatrix} 8 \\ -11 \\ -3 \end{pmatrix}$$

Определитель матрицы A равен -1, т.е. отличен от нуля, следовательно, решение единственно.

В результате работы программы был получен ответ:

$$x_1 = 2, \quad x_2 = 3, \quad x_3 = -1$$

Полученное решение совпадает с правильным ответом – алгоритм работает верно.

Следует отметить, что уже на первом шаге программа выбирает ведущим элементом число -3, поскольку его модуль максимален в столбце. Это уменьшает накопление ошибок округления и повышает точность вычислений.

Обработка вырожденной системы

Следующий эксперимент проводился для системы:

$$\begin{cases} x_1 + 2x_2 + 3x_3 = 6 \\ 2x_1 + 4x_2 + 6x_3 = 12 \\ x_1 - x_2 + x_3 = 2 \end{cases}$$

В этой системе матрица коэффициентов выглядит следующим образом:

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & -1 & 1 \end{pmatrix}$$

Сразу можно заметить, что вторая строка является первой, умноженной на два, следовательно строки линейно зависимы, следовательно $\det(A)=0$, а значит система вырожденна и не имеет единственного решения.

Во время выполнения метода Гаусса программа обнаруживает отсутствие ненулевого ведущего элемента и корректно завершает работу с сообщением о вырожденности матрицы.

Данный эксперимент подтверждает правильность проверки существования решения и корректную обработку подобных случаев.

Решение системы с малыми коэффициентами

В этом эксперименте рассматривалась система следующего вида:

$$\begin{cases} 0.0000001x_1 + x_2 = 1 \\ x_1 + x_2 = 2 \end{cases}$$

Её матрица коэффициентов выглядит следующим образом:

$$A = \begin{pmatrix} 0.0000001 & 1 \\ 1 & 1 \end{pmatrix}$$

Если бы метод Гаусса выполнялся без выбора ведущего элемента, то на первом шаге пришлось бы делить строку на число 0.0000001. После этого коэффициенты матрицы стали бы очень большими – порядка 10^7 .

При работе с числами с плавающей точкой подобная ситуация крайне опасна, поскольку ошибки округления начинают быстро накапливаться. Даже небольшая погрешность представления числа в памяти может привести к существенному искажению результата.

Однако в реализованной программе используется выбор ведущего элемента. Перед выполнением исключения алгоритм находит максимальный по модулю элемент текущего столбца и выполняет перестановку строк. В рассматриваемом примере ведущим элементом выбирается число 1 из второй строки, после чего вычисления выполняются устойчиво и без появления чрезмерно больших промежуточных коэффициентов.

Благодаря этому вычисления остаются точными, а программа успешно получает точное решение:

$$x_1 = 1, \quad x_2 = 1$$

Данный эксперимент показывает, что выбор ведущего элемента существенно повышает точность метода Гаусса и позволяет избежать проблем при работе с малыми коэффициентами.

Решение почти вырожденной системы

Для влияния ошибок округления была рассмотрена следующая система:

$$\begin{cases} x_1 + x_2 = 2 \\ 1.000000000000000001x_1 + x_2 = 2.000000000000000001 \end{cases}$$

Матрица коэффициентов:

$$A = \begin{pmatrix} 1 & 1 \\ 1.000000000000000001 & 1 \end{pmatrix}$$

Определитель этой матрицы равен -0.000000000000000001 . Фактически он не равен нулю, но очень к нему близок.

Такие системы называются плохо обусловленными, их особенность заключается в высокой чувствительности к ошибкам округления. Даже незначительное изменение коэффициентов или промежуточных вычислений может заметно изменить итоговый результат.

Точное решение системы имеет вид:

$$x_1 = 1, \quad x_2 = 1$$

Однако в результате работы программы был получен ответ:

$$x_1 = 0.8, \quad x_2 = 1.2$$

Несмотря на то, что алгоритм реализован корректно и использует выбор ведущего элемента, итоговое решение заметно отличается от точного.

Это связано с тем, что плохо обусловленные системы обладают высокой чувствительностью к вычислительным ошибкам. Даже крайне малые изменения коэффициентов матрицы или промежуточных результатов вычислений могут приводить к существенному изменению итогового решения.

Во время выполнения метода Гаусса многократно выполняются операции вычитания близких по значению чисел. В данном эксперименте:

1.0000000000000001-1

Результат такой операции оказывается чрезвычайно малым. При хранении вещественных чисел в памяти компьютера часть значащих цифр теряется, из-за чего даже малая неточность становится критичной.

В подобных системах даже незначительная погрешность промежуточных вычислений начинает усиливаться на последующих этапах метода Гаусса. В результате итоговый вектор решения может заметно отличаться от точного ответа, несмотря на математическую корректность самого алгоритма.

Данный эксперимент показывает, что использование выбора ведущего элемента значительно повышает точность метода, однако полностью устранить влияние ошибок округления при решении плохо обусловленных систем невозможно.

Итоги экспериментов

Проведённые эксперименты подтвердили корректность реализации метода Гаусса с выбором ведущего элемента и показали влияние вычислительных ошибок при работе с числами с плавающей точкой.

Эксперименты с невырожденными матрицами показали, что программа успешно решает стандартные системы линейных уравнений и получает корректные результаты.

Исследование вырожденных матриц подтвердило корректность проверки существования решения. Программа успешно определяет случаи линейной зависимости строк и завершает вычисления при невозможности получения единственного решения.

Эксперимент с малыми коэффициентами показал важность выбора ведущего элемента. Использование максимального по модулю элемента позволяет избежать деления на очень малые числа и значительно повышает точность вычислений.

Наиболее показательным оказался эксперимент с почти вырожденной системой. Было установлено, что даже при корректной реализации метода и использовании выбора ведущего элемента ошибки округления могут существенно влиять на итоговый результат. Это связано с

тем, что при работе с плохо обусловленными системами происходит потеря значащих цифр и накопление вычислительных погрешностей.

Таким образом, результаты экспериментов показали, что точность решения зависит не только от правильности алгоритма, но и от свойств самой системы. Использование выбора ведущего элемента повышает точность метода Гаусса, однако полностью устранить влияние ошибок округления при вычислениях с вещественными числами невозможно.

Заключение

В ходе выполнения работы был реализован алгоритм решения систем линейных уравнений методом Гаусса с выбором ведущего элемента. Для этого были разработаны шаблонные классы векторов и матриц, а также реализованы механизмы наследования, динамического выделения памяти и перегрузки операторов языка C++.

В процессе работы были рассмотрены основные теоретические аспекты линейной алгебры, связанные с решением систем линейных уравнений, преобразованием матриц и приведением системы к единичному виду. Особое внимание было уделено выбору ведущего элемента, который позволяет повысить точность вычислений и уменьшить влияние ошибок округления.

Полученные результаты показали, что метод Гаусса корректно работает для невырожденных систем и позволяет обнаруживать случаи, когда матрица является вырожденной. Также было установлено, что точность вычислений существенно зависит от структуры матрицы и особенностей представления вещественных чисел в памяти компьютера.

Проведённые эксперименты показали, что для хорошо обусловленных систем метод обеспечивает высокую точность вычислений, тогда как для почти вырожденных и плохо обусловленных матриц влияние ошибок округления становится значительно заметнее. Особенно ярко это проявляется при выполнении операций с очень близкими по значению числами, когда происходит потеря части значащих цифр.

Таким образом, можно сделать вывод, что точность решения системы линейных уравнений зависит не только от математической корректности алгоритма, но и от особенностей вычислений с числами с плавающей точкой. Использование выбора ведущего элемента позволяет значительно уменьшить влияние вычислительных ошибок и повысить точность метода Гаусса при решении систем линейных уравнений.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.

Приложение

```
template <typename T>
class Vector {
protected:
    T* vec;
    unsigned int size;
public:
    Vector() {
        size = 0;
        vec = nullptr;
    }
    Vector(int _size, T value = T()) {
        size = _size;
        vec = new T[size];
        for (int i = 0; i < size; i++) {
            vec[i] = value;
        }
    }
    Vector(const Vector& copy) { //Конструктор копирования
        size = copy.size;
        vec = new T[size];
        for (int i = 0; i < size; i++) {
            vec[i] = copy.vec[i];
        }
    }
    Vector& operator=(const Vector& rhs) { //Оператор присваивания
        if (this != &rhs) {
            delete[] vec;
            size = rhs.size;
            vec = new T[size];
            for (int i = 0; i < size; i++) {
                vec[i] = rhs.vec[i];
            }
        }
        return *this;
    }
    const T& operator[](int index) const { // Перегрузка оператора
        [], чтобы использовать запись v[i], где v - Vector<T>
        return vec[index];
    }
    T& operator[](int index) {
        return vec[index];
    }
    void PrintVector() {
        for (int i = 0; i < size; i++) {
            std::cout << vec[i] << " ";
        }
    }
    void PrintVector2() {
        for (int i = 0; i < size; i++) {
            std::cout << vec[i] << "\n";
        }
    }
};
```

```

    }
}
void FillVector() {
    for (int i = 0; i < size; i++) {
        std::cin >> vec[i];
    }
}
int GetSize() const{
    return size;
}
~Vector() {
    delete[] vec;
}
};

template <typename T>
class Matrix :public Vector<Vector<T>> {
public:
    Matrix(int n) :Vector<Vector<T>>(n, Vector<T>(n, T())) {};
    int GetSize() const {
        return this->size;
    }
    void PrintMatrix() {
        for (int i = 0; i < this->size; i++) {
            this->vec[i].PrintVector();// вызов PrintVector
для каждой строки
            std::cout << "\n";
        }
    }
    void FillMatrix() {
        for (int i = 0; i < this->size; i++) {
            this->vec[i].FillVector();// вызов FillVector для
каждой строки
        }
    }
};

template <typename T>
class SLU :public Matrix<T> {
public:
    SLU(int n) : Matrix<T>(n) {}
    Vector<T> Gauss(const Vector<T>& b) {
        int n = this->GetSize();
        Matrix<T> M = *this; // Копия матрицы
        Vector<T> x(n); // Копия правой части
        for (int i = 0; i < n; i++) {
            x[i] = b[i];
        }
        for (int k = 0; k < n; k++) {
            int number = k; //номер строки с максимальным элементом
            T maxelem = M[k][k]; // максимальный элемент
            if (maxelem < 0) maxelem = -maxelem;

            for (int i = k + 1; i < n; i++) {

```

```

        T elem = M[i][k];
        if (elem < 0) elem = -elem;
        if (elem > maxelem) {
            maxelem = elem;
            number = i;
        }
    }
    if (maxelem == 0) {
        std::cout << "Матрица вырождена. Решение не
единственно или не существует \n";
        return Vector<T>(0);
    }

    if (number != k) { // Меняем строку k и number местами
в матрице M и в правой части x
        for (int j = 0; j < n; j++) {
            T tmp = M[k][j];
            M[k][j] = M[number][j];
            M[number][j] = tmp;
        }
        T tmp = x[k];
        x[k] = x[number];
        x[number] = tmp;
    }

    T del = M[k][k]; // Делим всю СТРОКУ k на M[k][k]
(M[k][k] становится = 1)
    for (int j = 0; j < n; j++) {
        M[k][j] /= del;
    }
    x[k] /= del;

    for (int i = 0; i < n; i++) { //зануляем СТОЛБЕЦ k во
всех строках, кроме k
        if (i != k) {
            T agusha = M[i][k];
            for (int j = 0; j < n; j++) {
                M[i][j] -= agusha * M[k][j];
            }
            x[i] -= agusha * x[k];
        }
    }
    return x;
}
};

```